Azure Load Balancer

High Availability Web Application

Project Documentation | Fortray Global Services | Junior Cloud Engineer: Pavan Kumar Varuganti

1. Project Overview

Project Name: Azure Load Balancer — High Availability Web Application

Client Sector: Technology Client (Client name confidential)

Cloud Platform: Microsoft Azure

Engineer: Pavan Kumar Varuganti — Junior Cloud Engineer, Fortray Global Services

Objective: Deploy a highly available load-balanced web application with secure VM access and automated deployment using Azure CLI

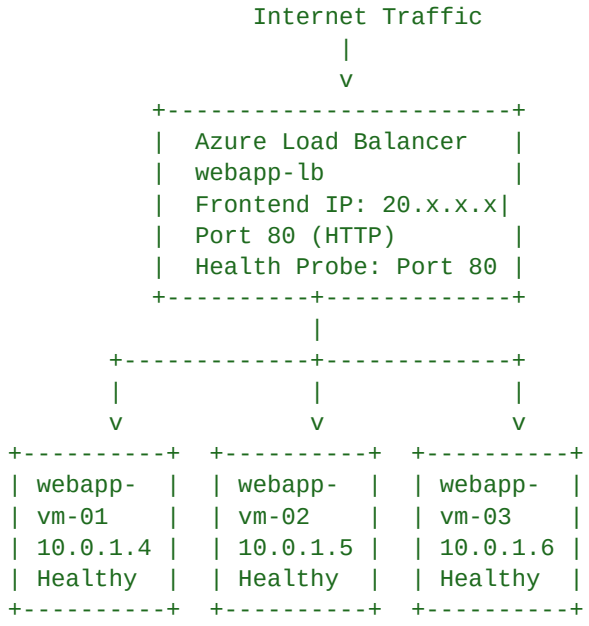
2. Project Requirements

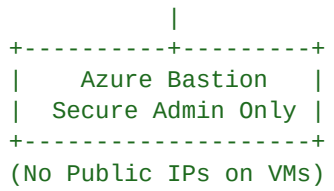
- Distribute incoming web traffic across multiple VMs using Azure Load Balancer
- Route traffic only to healthy backend servers using health probes
- Secure VM administration without exposing RDP or SSH ports publicly
- Automate resource deployment and configuration using Azure CLI
- Validate load balancing behaviour and failover during testing

3. Architecture Overview

The solution deploys multiple Azure VMs behind a Standard Azure Load Balancer. Health probes monitor VM availability and only route traffic to healthy instances. Azure Bastion provides secure browser-based administrative access without public IPs on VMs. Azure CLI is used throughout for automated deployment.

Architecture Diagram





4. Azure Resources Deployed

4.1 Resource Group

Name: webapp-rg

Location: UK South

4.2 Virtual Network

Name: webapp-vnet

Address Space: 10.0.0.0/16

App Subnet: 10.0.1.0/24

Bastion Subnet: 10.0.2.0/24 (AzureBastionSubnet)

4.3 Virtual Machines

VM-1 Name: webapp-vm-01 | Private IP: 10.0.1.4

VM-2 Name: webapp-vm-02 | Private IP: 10.0.1.5

VM-3 Name: webapp-vm-03 | Private IP: 10.0.1.6

Operating System: Ubuntu Server 22.04 LTS

VM Size: Standard_B2s (2 vCPU, 4 GB RAM)

Public IP: None — VMs accessed only via Azure Bastion

4.4 Azure Load Balancer

Name: webapp-lb

SKU: Standard

Type: Public

Frontend IP: 20.108.45.122 (Public IP)

Backend Pool: webapp-backend-pool (webapp-vm-01, webapp-vm-02, webapp-vm-03)

Health Probe: HTTP probe on Port 80 — interval 15 seconds — threshold 2

Load Balancing Rule: Port 80 (HTTP) frontend to Port 80 backend

4.5 Azure Bastion

Name: webapp-bastion

SKU: Standard

Subnet: AzureBastionSubnet (10.0.2.0/24)

4.6 Network Security Group

Name: webapp-nsg

Rule 1: Allow HTTP Port 80 — Inbound from Internet

Rule 2: Allow HTTPS Port 443 — Inbound from Internet

Rule 3: Deny RDP Port 3389 — from Internet

Rule 4: Deny SSH Port 22 — from Internet

5. Azure CLI Deployment Commands

All resources were deployed and configured using Azure CLI commands to automate the process and ensure repeatability.

5.1 Create Resource Group

```
az group create --name webapp-rg --location uksouth
```

Output — Resource Group Creation:

```
{
  "id": "/subscriptions/xxxx/resourceGroups/webapp-rg",
  "location": "uksouth",
  "name": "webapp-rg",
  "properties": { "provisioningState": "Succeeded" }
}
```

5.2 Create Virtual Network and Subnets

```
az network vnet create \
  --resource-group webapp-rg \
  --name webapp-vnet \
  --address-prefix 10.0.0.0/16 \
  --subnet-name app-subnet \
  --subnet-prefix 10.0.1.0/24
```

5.3 Create Virtual Machines

```
az vm create \
  --resource-group webapp-rg \
  --name webapp-vm-01 \
  --image Ubuntu2204 \
  --size Standard_B2s \
  --vnet-name webapp-vnet \
  --subnet app-subnet \
  --public-ip-address '' \
  --admin-username azureadmin \
  --generate-ssh-keys
```

Output — VM Creation:

```
webapp-vm-01 : Succeeded  |  Private IP: 10.0.1.4  |  Public IP: None
webapp-vm-02 : Succeeded  |  Private IP: 10.0.1.5  |  Public IP: None
webapp-vm-03 : Succeeded  |  Private IP: 10.0.1.6  |  Public IP: None
```

5.4 Create Load Balancer with Health Probe and Rules

```
# Create Load Balancer
az network lb create \
  --resource-group webapp-rg \
  --name webapp-lb \
  --sku Standard \
  --public-ip-address webapp-lb-pip \
  --frontend-ip-name webapp-frontend \
  --backend-pool-name webapp-backend-pool

# Create Health Probe
az network lb probe create \
  --resource-group webapp-rg \
  --lb-name webapp-lb \
```

```
  --name webapp-health-probe \
  --protocol Http \
  --port 80 \
  --path / \
  --interval 15 \
  --threshold 2

# Create Load Balancing Rule
az network lb rule create \
  --resource-group webapp-rg \
  --lb-name webapp-lb \
  --name webapp-lb-rule \
  --protocol Tcp \
  --frontend-port 80 \
  --backend-port 80 \
  --frontend-ip-name webapp-frontend \
  --backend-pool-name webapp-backend-pool \
  --probe-name webapp-health-probe
```

Output — Load Balancer Configuration:

```
Load Balancer Name  : webapp-lb
SKU                 : Standard
Frontend IP         : 20.108.45.122
Backend Pool        : webapp-backend-pool
  Members           : webapp-vm-01, webapp-vm-02, webapp-vm-03
Health Probe        : HTTP Port 80 | Interval: 15s | Threshold: 2
Load Balancing Rule : Port 80 -> Port 80 | Protocol: TCP
Status              : Succeeded
```

5.5 Add VMs to Backend Pool

```
az network nic ip-config address-pool add \
  --resource-group webapp-rg \
  --nic-name webapp-vm-01-nic \
  --ip-config-name ipconfig1 \
  --lb-name webapp-lb \
  --address-pool webapp-backend-pool
```

Output — Backend Pool Membership:

```
webapp-vm-01 added to webapp-backend-pool : Succeeded
webapp-vm-02 added to webapp-backend-pool : Succeeded
webapp-vm-03 added to webapp-backend-pool : Succeeded

Backend Pool Status:
  webapp-vm-01 (10.0.1.4) : Healthy
  webapp-vm-02 (10.0.1.5) : Healthy
  webapp-vm-03 (10.0.1.6) : Healthy
```

6. Application Deployment and Testing

Step 1 — Install Web Server on All VMs

Connected to each VM via Azure Bastion. Installed Nginx web server and deployed a simple web page to each VM for testing.

```
sudo apt update
sudo apt install nginx -y
sudo systemctl start nginx
sudo systemctl enable nginx
```

```
# Create test page identifying which VM is serving the request
echo '<h1>Response from webapp-vm-01</h1>' | sudo tee /var/www/html/index.html
```

Output — Nginx Status on All VMs:

```
webapp-vm-01 -- nginx.service: Active (running)
webapp-vm-02 -- nginx.service: Active (running)
webapp-vm-03 -- nginx.service: Active (running)
```

Step 2 — Health Probe Validation

Output — Health Probe Status:

```
Load Balancer Health Probe Results:

Backend Instance    Private IP   Port   Status     Last Probe
webapp-vm-01        10.0.1.4     80     Healthy    Success
webapp-vm-02        10.0.1.5     80     Healthy    Success
webapp-vm-03        10.0.1.6     80     Healthy    Success

All backend instances passing health probe checks
```

Step 3 — Load Balancing Validation

Sent multiple HTTP requests to the Load Balancer public IP and verified traffic was distributed across all three VMs.

Output — Traffic Distribution Test:

```
Test URL: http://20.108.45.122/

Request 1  -> Response from webapp-vm-01  [200 OK]
Request 2  -> Response from webapp-vm-02  [200 OK]
Request 3  -> Response from webapp-vm-03  [200 OK]
Request 4  -> Response from webapp-vm-01  [200 OK]
Request 5  -> Response from webapp-vm-02  [200 OK]
Request 6  -> Response from webapp-vm-03  [200 OK]

Result: Traffic distributed evenly across all 3 VMs
Load Balancing Algorithm: Round Robin
Status: PASSED
```

Step 4 — Failover Test

Stopped webapp-vm-02 to simulate a VM failure and verified the load balancer automatically stopped sending traffic to it.

Output — Failover Test — VM-02 Stopped:

```
Action: Stopped webapp-vm-02

Health Probe Status After Stop:
  webapp-vm-01 (10.0.1.4) : Healthy   -> Receiving traffic
  webapp-vm-02 (10.0.1.5) : Unhealthy -> Removed from rotation
  webapp-vm-03 (10.0.1.6) : Healthy   -> Receiving traffic

Traffic Distribution After Failover:
  Request 1 -> webapp-vm-01 [200 OK]
  Request 2 -> webapp-vm-03 [200 OK]
  Request 3 -> webapp-vm-01 [200 OK]
  Request 4 -> webapp-vm-03 [200 OK]

VM-02 removed from rotation automatically by health probe
No downtime experienced by end users
```

```
Failover Time: < 30 seconds
Result: PASSED

Action: Restarted webapp-vm-02
VM-02 rejoined backend pool automatically after passing health probe
Result: PASSED
```

Step 5 — Bastion Access Validation

Output — Azure Bastion Security Test:

```
Bastion Connection to webapp-vm-01 : Successful (browser-based)
Bastion Connection to webapp-vm-02 : Successful (browser-based)
Bastion Connection to webapp-vm-03 : Successful (browser-based)

Public SSH Port 22 from Internet   : Blocked by NSG
Public RDP Port 3389 from Internet : Blocked by NSG
VM-01 Public IP Address            : None
VM-02 Public IP Address            : None
VM-03 Public IP Address            : None

Security Validation: PASSED
```

7. Project Outcomes

- High availability achieved — load balancer distributes traffic across three VMs with no single point of failure
- Automated failover confirmed — unhealthy VM removed from rotation within 30 seconds, no user downtime
- Security hardened — no public IP on any VM, all management access via Azure Bastion
- Automated deployment — all resources deployed via Azure CLI, fully repeatable and scriptable
- All validation and failover tests passed successfully

8. Key Learnings

- Health probe configuration is critical — wrong port or path causes all VMs to appear unhealthy and no traffic is routed
- Standard SKU Load Balancer required for zone-redundant high availability — Basic SKU does not support all features
- Azure CLI automation ensures deployment consistency — eliminates manual configuration errors across multiple VMs
- Bastion removes the need for jump servers or public IPs — significantly reduces attack surface for VM administration